3-Tier Architecture

in E-Commerce Systems

A structured approach to building scalable, maintainable, and secure online retail platforms.

By,

Dwarkesh	23z431
Kapil Arif	23z432
Sudharsan	23z435
Vasanthkumaar	23z436
Thakshin Kumar T	22z266
Akash S	22z255

What is Multi-Tier Architecture?

The Problem with Monolithic Systems

Traditional monolithic applications bundle all logic — UI, processing, and data — into a single unit. As systems grow, this creates serious problems:

Difficult to Scale

One bottleneck can drag down the entire system. You can't scale one component independently.

Hard to Maintain

Any change risks breaking unrelated parts of the system. Teams step on each other.

Security Risks

UI directly touches the database, exposing sensitive data to potential attacks.

Deployment Issues

A small update requires redeploying the whole application, causing downtime.

Multi-tier architecture solves this by separating concerns into distinct, independently manageable layers.

From 1-Tier to 3-Tier: The Evolution

1-Tier

(Monolithic)

UI + Logic + Data

UI, Logic & Data all in one application. Simple but impossible to scale.

2-Tier

(Client-Server)

UI (Client)

Logic + Data (Server)

Client handles UI; Server handles logic + data. Better, but logic & data still mixed.

3-Tier

(Separated)

Presentation Tier

Application Tier

Data Tier

UI, Business Logic, and Data are fully separated. Each layer is independently scalable.

3-Tier Architecture is the industry standard for large-scale e-commerce systems.

3-Tier Architecture: The Big Picture

Presentation Tier

The User Interface Layer

Everything the customer sees and interacts with — browser, mobile app, search bar, product pages, checkout form.

Application Tier

The Business Logic Layer

The brain of the system — processes orders, validates payments, applies pricing rules, manages sessions and workflows.

Data Tier

The Storage Layer

Stores and manages all data — product catalogs, user accounts, inventory, orders, transactions — reliably and securely.

Key Principle: Each tier communicates only with its adjacent tier. Data never skips a layer.

Presentation Tier — The User Interface Layer

What is it?

- The Presentation Tier is the front-end of any e-commerce system. It is the only layer customers directly interact with — via web browsers or mobile apps.
- It collects user input, renders product information, handles navigation and checkout flows, and presents results back to the customer.

Product Discovery

Search bars, filters, product listings, sorting & category navigation.

Shopping Cart & Checkout

Cart management, address forms, order summary, payment UI.

Account Management

Login/signup, profile pages, order history, wishlist.

Multi-Platform Reach

Responsive web design + native iOS/Android apps.

Why the Presentation Tier Matters in E-Commerce

The Presentation Tier is the sole point of contact between the business and its customers. Its design and performance directly impact revenue, trust, and conversion rates.

Independent Scalability

During high traffic events (flash sales, peak hours), only the Presentation Tier needs to scale — more servers can be spun up without touching the logic or database layers.

Performance = Revenue

Studies show a 1-second delay in page load can reduce conversions by 7%. A fast, responsive UI directly increases sales. CDN delivery and caching are applied here.

Easy UI Updates

Redesigning the storefront, updating brand colours, or A/B testing a new checkout flow requires zero changes to the business logic or database — changes are isolated.

No Direct DB Access

The Presentation Tier never touches the database directly. All data requests go through the Application Tier, eliminating a major class of injection-based attacks.

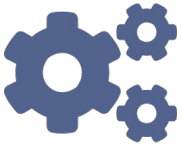
Global Accessibility

Multi-language support, accessibility standards (WCAG), and responsive design are managed in this tier — making the platform inclusive and globally reachable.

Separation of Concerns

Front-end developers can work independently from back-end teams. The UI can evolve rapidly using modern frameworks (React, Vue) without blocking server-side development.

Application Tier — The Business Logic Layer



What is it?

- The Application Tier is the intelligence of the system. It sits between the UI and the database, processing every user action with business rules.
- It never shows anything to the user directly and never stores data permanently — it purely orchestrates, validates, and coordinates.

Order Processing

Receives order requests, validates them, and coordinates fulfilment steps.

Payment Validation

Verifies payment details, interfaces with payment gateways, and handles fraud checks.

Inventory Management

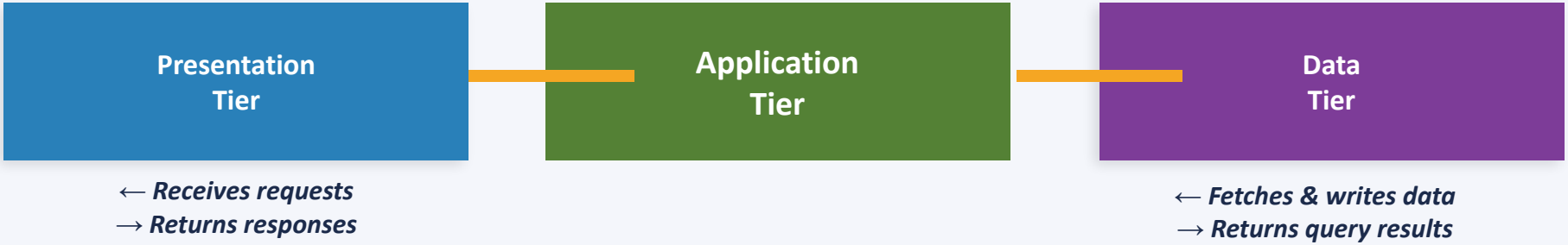
Checks stock levels in real time, reserves items, and triggers replenishment.

Pricing & Promotions

Applies discount codes, dynamic pricing rules, taxes, and shipping cost calculations.

Why the Application Tier is the Core of E-Commerce

Without a dedicated Application Tier, business logic would be scattered across the UI and database, making the system insecure, brittle, and impossible to evolve.



Centralised Business Rules

Discount logic, tax policies, and shipping rules live in one place. Updates affect all channels instantly — web, mobile, API.

Concurrency Handling

Handles thousands of simultaneous transactions with load balancing, queuing, and request routing — preventing race conditions on inventory.

Security Firewall

Acts as a gate — the UI can NEVER query the DB directly. All data access is mediated, logged, and controlled by this layer.

Integration Hub

Connects to payment gateways (Stripe, PayPal), shipping APIs (FedEx, UPS), and analytics — without the UI or DB knowing each other.

Data Tier — The Storage & Persistence Layer

What is it?

- The Data Tier is the permanent memory of the system. It stores everything the platform needs to operate — products, customers, orders, inventory, and transactions.
- It only responds to queries from the Application Tier and is never exposed to the outside world, providing a critical security boundary.

Product & Inventory DB

Stores SKUs, descriptions, pricing, stock levels — updated in real time as orders are placed.

Customer & Account Data

Profiles, addresses, credentials (hashed), purchase history, and personalisation data.

Order & Transaction Records

Every order placed, its status, payment confirmation, fulfilment history, and refunds.

Session & Cache Layer

Redis/Memcached caching for high-speed access to frequently queried data like product listings.

Why the Data Tier is the Backbone of E-Commerce

Every click, search, purchase, and review generates data. The Data Tier must be fast, consistent, and protected.

Relational DB (SQL)

MySQL / PostgreSQL

Used for:

Orders, customers, payments — structured transactional data with ACID compliance.

NoSQL DB

MongoDB / DynamoDB

Used for:

Product catalogs, user sessions, flexible schemas for evolving data structures.

Cache Layer

Redis / Memcached

Used for:

Lightning-fast reads for popular products, session tokens, and search results.

Search Engine

Elasticsearch

Used for:

Full-text search, faceted filtering, auto-complete — powers the product search bar.

Key Benefits of 3-Tier Architecture in E-Commerce

Each benefit maps directly to a real business need in e-commerce operations.

Scalability

Scale each tier independently. Add more app servers during a flash sale without touching the database.

Maintainability

Teams can update the UI, business rules, or DB schema independently without risk of breaking other layers.

Security

Database is never exposed to the internet. All access is mediated through validated application logic.

Performance

Caching at the data tier, load balancing at the app tier, and CDN at the presentation tier cut latency.

Reliability

Distributed infrastructure means one failed component does not bring down the entire platform.

Flexibility

Swap technologies in any tier (e.g. change DB, or switch to React) without rewriting the whole system.

REAL-WORLD CASE STUDY

Flipkart

3-Tier Architecture Powering

India's Largest E-Commerce Platform

Speaker 5: Real-World Application

Flipkart's 3-Tier Architecture in Practice

Presentation Tier

React Native (Android/iOS) · React.js (Web) · Akamai CDN

- App on your phone & website: product browsing, search, cart, checkout, order tracking — all customer-facing screens
- Flipkart Lite (progressive web app) serves low-bandwidth users in Tier-2/3 cities with the same back end
- All product images, banners, and assets delivered via Akamai CDN — fast load across every Indian city and town
- Separation benefit: Flipkart redesigned their checkout UI for Big Billion Days 2023 without touching any backend order logic

Application Tier

Java Microservices · Apache Kafka · Kubernetes · Razorpay/PhonePe/UPI APIs

- Core logic: product availability check, cart validation, order placement, payment processing, delivery slot allocation
- 100+ microservices handle search ranking, recommendations, fraud detection, and seller notifications independently
- Kafka event streams coordinate async tasks — SMS confirmations, invoice generation, warehouse pick instructions
- Kubernetes auto-scales this tier during Big Billion Days — crores of requests handled without touching the DB or UI

Data Tier

MySQL · Apache HBase · Elasticsearch · Redis · Apache Cassandra

- MySQL handles all transactional data — orders, payments, refunds — with ACID compliance so your ₹ is deducted exactly once
- Redis caches 'Deals of the Day' and product listing pages — sub-millisecond reads for crores of simultaneous visitors
- Elasticsearch powers Flipkart's search with spelling correction, filters, and personalised ranking across 15 crore+ listings
- Cassandra stores massive-scale event logs, clickstream data, and seller analytics — completely isolated from customer data

How 3-Tier Architecture Solved Flipkart's Challenges

CHALLENGE: Big Billion Days: 10 Crore Visitors in 1 Hour

✓ SOLUTION: Application Tier scales independently

During Big Billion Days, Flipkart receives 10 crore+ visitors within the first hour. Only the Application Tier (Kubernetes pods) auto-scales — the database and the storefront remain completely unchanged. Zero downtime for any seller or buyer.

CHALLENGE: 15 Crore+ Product Listings, All Searchable

✓ SOLUTION: Data Tier with Elasticsearch + HBase

A single product (e.g. a phone) may have 500+ seller listings with different prices, conditions, and delivery timelines. Elasticsearch indexes all of them and returns ranked, filtered results in milliseconds without touching the main transaction DB.

CHALLENGE: App, Website, Flipkart Lite — All Different UIs

✓ SOLUTION: Presentation Tier decoupled via API

Flipkart's Android app, iOS app, full website, and Flipkart Lite PWA are four separate Presentation Tiers — all connecting to the same Application and Data Tiers via API. A UI redesign never requires a single change to order logic or the database.

CHALLENGE: UPI/EMI Payment Must Never Charge Twice

✓ SOLUTION: Application Tier + ACID-compliant Data Tier

If a user's phone loses signal mid-payment, the Application Tier checks the database for transaction status before retrying. MySQL's ACID compliance guarantees the ₹ is deducted exactly once — even across network failures during peak sale traffic.

REAL-WORLD CASE STUDY

Zomato

3-Tier Architecture Powering India's #1 Food Delivery Platform

Speaker 5: Real-World Application

Zomato's 3-Tier Architecture in Practice

Presentation Tier

React Native (Android/iOS) · React.js (Web) · Cloudflare CDN

- App on your phone: menu browsing, restaurant search, cart, checkout, live delivery tracking map
- Storefront API enables the web version and partner integrations — same back end, different UI
- All images, menus and assets served via Cloudflare CDN — fast load regardless of your city
- Separation benefit: Zomato redesigned their entire app UI in 2023 without touching a single backend system

Application Tier

Ruby on Rails · Go Microservices · Kafka · Kubernetes · Razorpay/PayTM APIs

- Core logic: order validation, delivery partner assignment, ETA calculation, surge pricing, fraud checks
- Microservices in Go handle payments, notifications, and restaurant coordination independently
- Kafka processes async tasks — SMS, push notifications, restaurant KOT printing — without blocking orders
- Kubernetes auto-scales this tier on Diwali/IPL nights — 10x pods spin up without touching the DB or app

Data Tier

MySQL · Redis · Elasticsearch · Cassandra · Google Cloud Storage

- MySQL stores all orders, payments, and customer profiles with full ACID compliance — your ₹ is deducted exactly once
- Redis caches 'Top Restaurants Near You' results — sub-millisecond reads without hitting the main database
- Elasticsearch powers the search bar with typo-tolerance ('Chiknnn Tikka' still finds Chicken Tikka)
- Object storage holds all restaurant photos and menus at scale — completely isolated from the internet

How 3-Tier Architecture Solved Zomato's Challenges

CHALLENGE: New Year's : 10x Orders in 10 Minutes

✓ SOLUTION: Application Tier scales independently

On Dec 31, Zomato sees a 10x order surge. Only the Application Tier (Kubernetes pods) auto-scales — the database and the app remain completely unchanged. Users experience zero slowdown or crash.

CHALLENGE: 5 Lakh Restaurants with Different Menus Daily

✓ SOLUTION: Data Tier with NoSQL + Elasticsearch

A pizza shop has 40 variants; a biryani stall has 5; a cloud kitchen changes its menu every day. NoSQL stores this flexible data. Elasticsearch makes everything searchable with typo-tolerance and cuisine filters.

CHALLENGE: Android App, iPhone App, Website — All Different

✓ SOLUTION: Presentation Tier decoupled via API

Zomato's Android app, iOS app, and website are three separate Presentation Tiers — all connecting to the same Application and Data Tiers via API. A redesign of the app does not require a single change to order logic or the database.

CHALLENGE: UPI Payment Must Never Double-Debit You

✓ SOLUTION: Application Tier + ACID Data Tier

If your phone crashes after tapping 'Pay', the Application Tier checks the database to see if the transaction was committed before retrying. MySQL's ACID guarantees your ₹450 is deducted exactly once — every time, without exception.

Key Takeaways

01 **Separation of Concerns**

Each tier has one responsibility. This makes large e-commerce systems manageable, testable, and evolvable.

02 **Independent Scalability**

E-commerce traffic is unpredictable. 3-tier architecture lets you scale precisely where the load is — not everywhere.

03 **Security by Design**

The database is never internet-facing. The Application Tier acts as an auditable, controllable security boundary.

04 **Real-World Proven**

Shopify, Flipkart, eBay, and every major e-commerce platform uses multi-tier architecture at its core.

3-Tier Architecture is not just a design pattern — it is the foundation that makes modern e-commerce possible.

Thank You !